

Self-Guided Lab

Peterson's Generalised Algorithm in CSP-M

6CM504 — Concurrency and Communication

Exercise 6, Task 2 — Preparation

How to use this lab

This worksheet is designed to be worked through independently, at your own pace. It builds up the concepts and CSP-M techniques you need step by step, from simple register processes all the way to a complete mutual exclusion model.

Each section has tasks that ask you to write and test small CSP-M scripts in FDR. By the end, you will have all the building blocks needed for Exercise 6, Task 2.

Tip

Keep FDR open alongside this document. Type each code example in, run it, and experiment with modifications before moving on.

Estimated time: 2–3 hours across 1–2 lab sessions

Part 1: Modelling shared memory

In CSP, there is no concept of shared memory. Processes communicate only through synchronised events (channels). To model a shared variable, we create a process that holds a value and offers read and write events.

1.1 A simple register

Let's start with the simplest possible shared variable: a register that holds a single integer value.

Task 1.1

Create a register process

Type the following into a new CSP-M file and load it in FDR. Use Probe to step through traces.

```
-- A register that holds value v
-- It can be read (returning v) or written (updating to a new value)

datatype RegID = Reg0 | Reg1 | Reg2
Values = {-1..2}

channel read  : RegID . Values
channel write : RegID . Values

REG(id, v) = read.id.v -> REG(id, v)
            []
            write.id?new_v -> REG(id, new_v)
```

Check

In Probe, can you perform `read.Reg0.(-1)`? What about `read.Reg0.0`? Why does only one of these succeed when the register starts at -1?

1.2 Testing the register

Before using registers in a larger system, let's verify they work correctly.

Task 1.2

Write a test process

Create a process that writes a value, then reads it back. Compose it with the register and check the trace in Probe.

```
-- A simple test: write 1, then read it back
TEST_WRITER = write.Reg0.1 -> read.Reg0.1 -> SKIP

-- Compose the test with the register
-- They synchronise on {| read.Reg0, write.Reg0 |}
TEST_SYSTEM = TEST_WRITER
               [| {| read.Reg0, write.Reg0 |} |]
               REG(Reg0, -1)
```

Check

What happens if you try to read a value that the register doesn't hold? For example, if you change `read.Reg0.1` to `read.Reg0.0` — does the system deadlock?

1.3 Multiple registers

Peterson's algorithm needs multiple registers. Let's compose several together.

Task 1.3

Create a bank of registers

Create three level registers (for 3 processes) and two `last_to_enter` registers (for 2 levels). Compose them in interleave.

```
-- Three level registers, initially -1
LEVELS = REG(Reg0, -1) ||| REG(Reg1, -1) ||| REG(Reg2, -1)

-- For last_to_enter we need separate channel names
-- to avoid confusion with level registers
datatype LteID = Lte0 | Lte1
channel read_lte : LteID . {0..2}
channel write_lte : LteID . {0..2}

LTE_REG(id, v) = read_lte.id.v -> LTE_REG(id, v)
                []
                write_lte.id?new_v -> LTE_REG(id, new_v)

LTES = LTE_REG(Lte0, 0) ||| LTE_REG(Lte1, 0)
```

Hint

Using separate channel names (`read` vs `read_lte`) makes it much easier to control synchronisation later. You know exactly which channels each component uses.

Part 2: Understanding Peterson's algorithm

Before writing CSP-M, you must deeply understand what the algorithm does. This section walks you through the logic.

2.1 The filtering principle

With N processes and $N-1$ levels, each level acts as a gate that filters out exactly one process. At each level:

- A process announces its level and marks itself as the last to enter
- It then spin-waits until either someone else arrives last, or all others drop below this level
- The "last to arrive" is the loser — everyone else passes through

2.2 Pseudocode walkthrough

Here is the algorithm for process i (with N processes total):

```

FOR l = 0 TO N-2:
    level[i] := l           // announce: I want level l
    last_to_enter[l] := i   // announce: I was last here

    WHILE last_to_enter[l] == i // am I still the last?
        AND exists k != i     // is anyone else
            where level[k] >= l: // at my level or above?
            DO nothing          // yes – keep waiting

<< critical section >>

level[i] := -1             // release: I'm done

```

2.3 The two escape conditions

Process i escapes the WHILE loop when EITHER condition becomes false:

Escape 1: Someone else arrived last

`last_to_enter[l] != i`

Escape 2: Nobody else is competing

`no k != i where level[k] >= l`

Another process wrote its ID to `last_to_enter[i]`, overwriting yours. You're no longer the loser.

All other processes are either below your level or have finished (`level[k] = -1`). The path is clear.

**Task
2.1****Trace through by hand**

Take 3 processes (P0, P1, P2). On paper, trace through an interleaving where P0 and P1 both try to enter the CS at the same time, while P2 is idle. Track the values of `level[0..2]` and `last_to_enter[0..1]` at each step.

Hint

Start with: `level = [-1, -1, -1]`, `lte = [0, 0]`. When P0 writes `level[0]=0` and `lte[0]=0`, then P1 writes `level[1]=0` and `lte[0]=1`, what happens to P0's while loop?

Part 3: Building the CSP-M model

Now we translate each part of Peterson's algorithm into CSP-M, step by step.

3.1 Modelling the spin-wait

The WHILE loop is the hardest part. In CSP, a loop becomes recursion. The process reads the relevant variables and decides whether to keep spinning or advance.

For the spin-wait at level l , process i must check:

1. Read `last_to_enter[l]` — is it still me?
2. Read `level[k]` for all $k \neq i$ — is anyone at level l or above?
3. If both conditions are true: recurse (keep waiting)
4. If either is false: proceed to next level

Task 3.1

Model the spin-wait check

Write a CSP-M process that reads the necessary variables and decides whether to keep spinning. Think about: how do you check "exists $k \neq i$ where `level[k] >= l`"?

Hint

You can read all the level registers one at a time. You don't need to read them atomically — Peterson's algorithm is designed to work even when reads are interleaved with other processes' writes. After reading all values, use an if-then-else to decide.

Warning

Don't try to model the entire algorithm at once! Get the spin-wait logic working for a single process at a single level first, then generalise.

3.2 Modelling one level

A single iteration of the FOR loop (for process i at level l) has this structure:

```
-- Pseudocode for one level iteration:
-- 1. Write level[i] := l
-- 2. Write last_to_enter[l] := i
-- 3. Spin-wait until condition is false
-- 4. Continue (to next level or CS)
```

**Task
3.2****Model a single level gate**

Write a process `GATE(i, l, continue)` that performs steps 1–3 and then becomes the process "continue". The `continue` parameter lets you chain gates together.

Hint

Think of it as:

```
GATE(i, l, cont) = write.RegI.l -> write_lte.LteL.i -> SPIN(i, l, cont)
SPIN(i, l, cont) = <read and check> if should_wait then SPIN(...) else
cont
```

3.3 Chaining levels together

With $N=3$ processes, you need $N-1=2$ levels. Process i goes through Gate 0, then Gate 1, then enters the critical section.

**Task
3.3****Chain two gates and add CS events**

Define a `PROC(i)` that passes through Gate 0 and Gate 1, then performs `enter_cs.i`, then `exit_cs.i`, then sets `level[i] = -1` and loops.

```
channel enter_cs, exit_cs : {0..2}

-- Structure:
-- PROC(i) = GATE(i, 0,
--           GATE(i, 1,
--             enter_cs.i -> exit_cs.i ->
--             write.RegI.(-1) -> PROC(i)))
```

Tip

You may need to parameterise which `RegID` corresponds to process i . One approach: define a function or use indexed channel names like `read.i.v` and `write.i.v`.

Part 4: Composing the system

Now you put everything together: the process definitions, the registers, and the synchronisation.

4.1 The overall architecture

Your system consists of:

- PROC(0), PROC(1), PROC(2) — interleaved with each other
- LEVELS — the three level registers
- LTES — the two last_to_enter registers
- All synchronised on the shared read/write channels

```
-- The shared channel set
SHARED = {| read, write, read_lte, write_lte |}

-- All processes interleaved
PROCS = PROC(0) ||| PROC(1) ||| PROC(2)

-- All registers interleaved
REGS = LEVELS ||| LTES

-- System: processes synchronise with registers on shared channels
SYSTEM = PROCS [| SHARED |] REGS
```

Task 4.1

Load and explore in Probe

Assemble the full system and load it in FDR. Use Probe to step through traces. Can you get a process to enter the critical section? Can you get two processes through?

Warning

If FDR says your model has too many states, check your value ranges. Do your Values really need to go beyond $\{-1..2\}$? Smaller domains = dramatically smaller state spaces.

Part 5: Verification

The goal of modelling in CSP is to verify key properties of the system. For this model, the main properties are mutual exclusion, deadlock freedom, and divergence freedom. These properties are related but distinct, so they should be interpreted carefully.

5.1 Mutual exclusion (safety)

The specification says: at most one process can be in the critical section at any time. In CSP terms: between any two `enter_cs` events, there must be an `exit_cs`.

```
-- Specification: one-at-a-time
MUTEX_SPEC = enter_cs?p -> exit_cs.p -> MUTEX_SPEC

-- Hide everything except enter/exit from the system
SYSTEM_CS = SYSTEM \\ diff(Events, {| enter_cs, exit_cs |})

-- Traces refinement: SYSTEM can only do what SPEC allows
assert MUTEX_SPEC [T= SYSTEM_CS
```

Task 5.1

Check mutual exclusion

Add the assertion above to your script and run it in FDR. It should pass. If it fails, FDR gives you a counterexample trace — use it to find the bug!

5.2 Deadlock freedom

The model should be deadlock free: the overall system should never reach a state where no further event is possible. Note that this does not by itself guarantee that every individual process always makes progress.

```
assert SYSTEM :[deadlock free]
```

Task 5.2

Check deadlock freedom

Run this assertion in FDR. If it fails, inspect the trace to see which combination of process states and register values causes the whole model to stop.

5.3 Divergence freedom

A divergence is an infinite sequence of hidden (τ) actions that prevents observable progress. Divergence freedom means the model does not contain such infinite internal spinning. This is especially relevant to the spin-wait loop.

```
assert SYSTEM :[divergence free]
```

**Task
5.3****Check divergence freedom**

Run this assertion in FDR. If divergence is reported, check whether your spin-wait loop has been turned into hidden/internal activity. Keep the read events in the loop visible unless you have a good reason to hide them.

Tip

If divergence checking takes too long, it could indicate your state space is too large. Consider reducing value ranges or testing with 2 processes first before scaling to 3.

Warning

Divergence freedom is not the same as bounded waiting (starvation freedom). Bounded waiting is a stronger fairness property: it means a process that requests entry cannot be postponed forever. The divergence check alone does not prove bounded waiting. If you want to establish bounded waiting, that requires a separate argument or specification.

Part 6: Common problems and solutions

Problem	Likely cause and fix
FDR won't load the file	Syntax error in CSP-M. Check channel declarations, missing arrows (->), or mismatched brackets. FDR's error messages point to the line.
Immediate deadlock	Your processes and registers aren't synchronising properly. Check that the channel names in your process match the register's channels exactly.
Mutual exclusion fails	The spin-wait logic is wrong. Check both conditions carefully. A common mistake is checking OR instead of AND.
State space explosion	Your value ranges are too large. With N=3, level values only need {-1, 0, 1} (not {-1..10}). Process IDs need {0, 1, 2}.
Divergence detected	Your spin-wait uses hidden/internal events. Make sure the read events in the loop are visible (not hidden by the hiding operator).
One process appears to wait forever	Check whether the last_to_enter mechanism and level comparisons are modelled correctly. Use Probe to inspect the interleaving.